2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement

What if I had no smells?

Davide Falessi
Dept. of Computer Science and Software Engineering
California Polytechnic State University
San Luis Obispo, CA, USA
dfalessi@calpoly.edu

Barbara Russo
Faculty of Computer Science Free University of Bozen-Bolzano,
Bolzano, Italy USA
Barbara.Russo@unibz.it

Kathleen Mullen
Keymind A Division of Axiom Resource Management, Inc.
Falls Church, VA, USA
kmullen@keymind.com

***Abstract*— What would have happened if I did not have any code smell? This is an interesting question that no previous study, to the best of our knowledge, has tried to answer. In this paper, we present a method for implementing a what-if scenario analysis estimating the number of defective files in the absence of smells. Our industrial case study shows that 20% of the total defective files were likely avoidable by avoiding smells. Such estimation needs to be used with the due care though as it is based on a hypothetical history (i.e., zero number of smells and same process and product change characteristics). Specifically, the number of defective files could even increase for some types of smells. In addition, we note that in some circumstances, accepting code with smells might still be a good option for a company.**

Keywords: code smells, machine learning, software estimation, technical debt.

1 INTRODUCTION

Code bad smells, or shortly smells, are code artifacts that are symptoms of poor design and implementation choices [1] [2] [3]. There are several smell types [1], such as large classes (aka God classes), code clones, low comment frequency, and non-intuitive variable naming. Smells can leave in the code some "technical debt" (i.e., not-quite good code left in the released software) that can eventually lower the quality of the product. Whether smells really affect such quality is still an open issue, though [4][5]. The effects of smells have also been analyzed to understand how and when refactoring (e.g., removing existing smells) occurs [6].

A large amount of research has focused on the effects of smells on code or process quality factors, such defectiveness or effort, with methods based on mining process or product history (e.g., mining software repository) [7]. Such approaches aim to investigate what *has happened* in order to eventually predict what *will happen* (e.g., Are smells correlated with product quality?). However, they do not investigate what *would have happened* in the case of different hypothetical histories (e.g., What would be the quality of the code if smells had been avoided?).

One way to evaluate hypotheses different from the ones available in history is controlled experiments [4] [8] [9] [10] for which different hypotheses are observed on different samples of subjects (e.g., the same maintenance activity is performed in the absence or presence of smells). However, controlled experiments are very expensive and can hardly deal with multivariate problems (e.g., they can observe a limited number of smell types [11] [4]), a large number of subjects (e.g., [8] [4]), or several maintenance activities (e.g., [12] [4]).

Regardless, the techniques to analyze the history of data or results of controlled experiments are often too complex or not actionable to be fully understood and easily adopted by practitioners in their decision making process. Such techniques often use statistical instruments that practitioners find impractical, hard to interpret and use when they need to trade-off among business goals such as cost, quality, and time-to-market. Thus, they often rely on black box tools that enable them to make their decision quickly, with little technical knowledge of the actual statistical techniques implemented by the tools. In particular, several static analyzers exist to support management in the understanding of the technical debt of products under development [12], like the open-source software SonarQube[1]. These tools can analyze the compliance of a software project with hundreds of different smell types using quality rules. Figure 1 shows an excerpt of a smell as reported by SonarQube[1] for line 56 of the "SSLAuthenticator.java" class of the Apache Tomcat system (see online[2] for the full SonarQube output). The smell in Figure 1 is called "Avoid commented-out lines of code" and states that code should not be commented out, it should be simply removed.

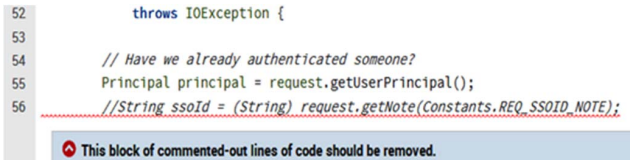


Figure 1: Example of a smell as reported by SonarQube.

Existing tools may also support the use of "quality gates." Such gates prevent the acceptance of push requests of code commits that are signaled to contain smells. The practitioner can decide to set such gates for all or some types of smells. Such a decision is left to the subjectivity of the practitioner. To our knowledge, there is no research that investigates whether avoiding smells (i.e., by setting gates in analyzers) can give decision makers some kind of actionable estimation of the quality of their software product.

[1] http://www.sonarqube.org/
[2] http://nemo.sonarqube.org/dashboard/index?id=Apache

978-1-5090-4039-1/17 $31.00 © 2017 IEEE
DOI 10.1109/ESEM.2017.14

IEEE computer society

Authorized licensed use limited to: Universita degli Studi di Roma Tor Vergata. Downloaded on April 16,2025 at 09:22:19 UTC from IEEE Xplore. Restrictions apply.

Thus, the aim of this work is *to devise a method to estimate the number of defective files* (i.e., files with at least one defect) *if smells are avoided*.

Our method is based on a What-if analysis. Specifically, we define a What-if scenario (i.e., if smells are avoided), prepare data accordingly, learn classifiers on actual data, and estimate file defectiveness on synthetic data obtained if the code had no smells.

Our method is applied on data collected from five different industrial projects and 106 different types of smells of one company called Keymind. To increase the generalization power of our work, we also provide a set of *assumptions* under which our method can be replicated on different data.

To explain the motivation of our work, we first report the common understanding of the relation between smells and code quality (Section 2 Intuition) and then the perspective of the manager of the company that kindly supplied the data for this study (Section 3 Industry Perspective).

The work finally reflects on the results and warns practitioners not to use any estimation method without investigating the nature and the reason for smells in their specific source code.

2 Intuition

The intuition behind this work is that the presence of smells has little observable effect on maintenance effort [4], but its removal decreases those measures that are positively correlated with code quality and actually reduces maintenance effort (e.g., churn) [4] [5]. For instance, the absence of the smell types "low comment frequency" and "non-intuitive variable naming" would make the code easier to understand and thus, eventually prevents defects in files. This indirect impact on code quality and the prompt availability of data analytics tools (e.g., SonarQube) may induce managers to use off-the-shelf instruments of prediction in the evaluation of the quality of their products.

A typical approach of decision makers is to perform a "What-If Analysis" and build "What-If Scenarios," under which they evaluate their business. A simple, commonly used "What-If Scenario" technique changes values of the variables of interest to see how those changes will affect the possible results [13]. Such easy-to-use instruments can give managers the illusion of control over their business without warning them of the risk to overestimate its effectiveness. Nonetheless, managers report it beneficial for their decision performance [13]. Thus, how can we support managers in using such a tool but in a more informative way?

In this work, we apply the "What-If Scenarios" technique to the estimation of defective files. Specifically, we change to zero the original number of code smells in non-linear models estimating the number of defective files from process and code change variables and pose the following question:

Does the number of defective files decrease if code smells are avoided and the product / process characteristics are kept invariant?

Statistical prediction models are then trained on the original datasets and further used in the two datasets (synthetic and original) to estimate defective files. It is worth noticing that such a scenario is different from the most frequently investigated case in which smells are actually removed (e.g., [14]) since our scenario does not require any intervention within the actual code.

A discussion and few recommendations are then drawn for the benefit of managers that aim to apply "What-If Scenarios" technique in a similar context of application and for similar industry goals.

3 The Industry Perspective

In this section, we describe the context of this study and the perspective and experience of the company and its manager in adopting data analytics tools to avoid or remove code smells.

3.1 Industry Context

Our industry context is Keymind, a CMMI Level 5 organization. Keymind is the technology and creative services division of Axiom Resource Management, a professional consulting firm based in Falls Church, Virginia. Keymind provides software development, strategic marketing, and user focused services to both commercial and government customers. The Keymind staff is comprised of award-winning leaders in interactive media who are widely recognized for distinctive website designs. Keymind focuses on a user-friendly experience, accessibility, and client-focused creative and technical support. It is a small to medium size organization of about 25-30 people and was successfully re-appraised at Maturity Level (ML) 5 of the Capability Maturity Model Integration for Development (CMMI-DEV), version 1.3 in September, 2015.

Keymind relies heavily on data collected from its suite of development environment tools to support quantitative project management and organizational decision-making. Their development environment tool suite includes: 1) JIRA, a web-based issue-tracking tool; 2) Subversion (SVN), integrated with JIRA, for configuration management including source code and documentation control; 3) Confluence, a wiki that serves as a process asset library and knowledge sharing repository; 4) Jenkins for supporting a continuous integration and unit testing automation; and 5) SonarQube for managing technical debt.

The development tools suite and Keymind's high maturity processes provide a broad range of data including defects that have been generated, and associated, with components, given commits that have been made to the SVN repository, as well as smells identified via SonarQube. One primary goal at Keymind is to make effective decisions related to components that need to be refactored (to remove technical debt) given the smells identified by SonarQube

3.2 Avoiding versus removing smells

One important practice in the context of technical debt management is the use of quality gates, i.e., rejecting all the code (commits) that do not pass through some sort of minimum threshold on smells. For instance, it is possible to enforce that new code will have no smell or, to be more flexible and customizable to the business constraints at hand, no smell types

Authorized licensed use limited to: Universita degli Studi di Roma Tor Vergata. Downloaded on April 16,2025 at 09:22:19 UTC from IEEE Xplore. Restrictions apply.

of a given priority level. This approach gives Keymind the ability to avoid increasing technical debt over time and also to carefully define the maximum amount of technical debt that is tolerable given the specific time and effort constraints of the release under development.

Specifically, for one reason or another, when people reason about smells, they think first about how to remove them via refactoring and often consider several very hard questions such as who should do the refactoring, how to get the required effort and time, and what the possible drawbacks may be. Alternatively, setting the quality gate is pretty straightforward, and it avoids technical debt accumulation in the first place, without wondering how to remove it once it exists. This is why in this paper, we aim at understanding whether we should avoid smells a priori rather than taking the common approach of refactoring (i.e., which smells should be removed once they exist in the code).

Thus, one of the main advantages of avoiding versus removing smells is that smells do not accumulate over time and that it avoids all the problems that refactoring could have, such as injecting defects (without adding any new functionality). The disadvantage of avoiding versus removing smells is that it requires some technology to be in place (to reject the smelly committed code) and it is applicable only to new code. Moreover, avoiding smells a priori might prevent meeting important business goals such as time to market. Therefore, some smells can be accepted in some circumstances.

3.3 The institutionalization challenge

Keymind installed SonarQube in June 2014 after customizing the rules prioritizations, on April 2014-June 2014 [15]. Moreover, in Feb 2015, Keymind implemented a SonarQube plug-in to check the compliance of projects to two reference architectures we called "Pure .NET" and "MVC/Entity Framework .NET".

In order to facilitate SonarQube usage, several senior Keymind staff attended the latest SonarQube workshop in June 2016. According to the manager, the workshop was useful for all those who attended it and reinforced the perceived value of using it.

In the near future, Keymind is going to set up a small team to re-review its customization of Sonar rules, including those related to architecture. Since 2014, the Keymind technical team has experienced some turnover and junior developers are now more senior, as a result, viewpoints have changed regarding some of the customizations made to Sonar's rule prioritizations. For example, in a recent code review, the team reported that keeping commented-out code is not a good thing, especially the longer it stays in the code. During its customization a few years ago, the technical team had lowered the priority of this SonarQube rule "Avoid commenting out code" but now Keymind plans to increase the priority back to SonarQube's default priority.

4 Related work

Sjøberg, et al. [4] performed the first controlled experiment with professionals investigating the impact of 12 smell types (called smells) on maintenance effort. Results showed that size impacts maintenance effort more than the 12 investigated smell types. Specifically, no smell types resulted as being significantly associated with increased effort after they adjusted for file size and the number of changes.

Yamashita and Moonen [11] reported an empirical study on the effect of 12 smell types with respect to maintenance issues. Their main finding is that there is an important interaction effect among smells.

Griffith and Izurieta [16] performed a simulation study for investigating the impact on understandability of eight different types of class grime, which is a type of smell that occurs when classes of design pattern have extraneous attributes or methods that confuse the intended design. Results showed that all the eight different types of class grime significantly impacted understandability.

A lot of further research effort has been spent investigating and enhancing 1) humans' performances in refactoring activities [6] [17] [18] [19] [20], 2) accurate identification of code smells [21] [22] [23], and 3) importance of removing the smells [24] [25] [26]. For example, Liu et al. [27] suggest a strategy to avoid the influence of code smells software. The authors claim up to 17.64–20% efforts can be reduced if the presence of bad smells is detected in time.

Yamashita [8] analyzes the effects of twelve smell types on problems perceived by developers. Her results show that the smell type called Interface Segregation Principle (a.k.a. ISP violation) is strongly correlated with maintenance problems. Thus they recommend removing this type of smell to ease future maintenance efforts.

Abbes et al. [9] analyze the effects of two smell types, called Blob and Spaghetti Code, on program comprehension. Their results show that each smell has a small impact on comprehension when existing in isolation with the other smell type. However, the negative impact on comprehension is significant when both smell types exist. Thus, their result suggests that the impact of smells on maintenance is not linear.

Deligiannis et al. [10] analyze the effects of one smell type, called God Class, on maintainability. Their results show that the existence of God Class significantly and negatively impacts maintainability and reuse.

Finally, in a previous study [15] we use the same dataset we use in this study and we showed that classes with a low number of smells per LOC are clearly less defect prone than classes with a high number of smells per LOC. Thus in the past we focused on assessing if classes with a higher number of smells had more defects. Here we are interested in estimating how much the number of defective classes would have decreased if smells had been avoided.

To our knowledge, there is not much literature on performing a What-If analysis in software engineering. The only example we were able to find is the work of Bird and Zimmerman [28], who present a study on the use of branches in software development. Bird and Zimmerman proposed a What-If Analysis aimed at assessing alternative scenarios for branch structures.

Authorized licensed use limited to: Universita degli Studi di Roma Tor Vergata. Downloaded on April 16,2025 at 09:22:19 UTC from IEEE Xplore. Restrictions apply.

5 Study Design

In this section, we describe our method to estimate the number of defective files if such files had no smells. The method consists of five steps: 1) Metric selection, 2) Metric collection, 3) Dataset manipulation, 4) Model selection, and 5) Estimation results.

5.1 Data description

The method is applied on the historical analysis of the JIRA and SVN repositories of ten years of five projects of Keymind. The main programming language used is C# supported by .NET framework. Each project has an average of 11 releases per year and each release has an average of two internal releases before the code is provided to the client. Each release lasts an average of 31 days and it has an average of 113 files. In a previous study [15] we report additional details about the dataset.

5.2 Metric selection.

Besides the initial size and number of smells and the final number of defective files, we selected those metrics that are able to describe the maintenance activities of a file during a release. These metrics characterize both the process and the product. We selected 16 change metrics that have been extensively used for defect estimation [29]. Table 1 reports such metrics.

5.3 Metrics collection.

With SonarQube, which is able to identify 1297 different smell types, we collected the number of smells in a file per single revision. In total, we had 6912 smells covering 106 different smell types over 1348 files. Smells are typically measured at the beginning of a release and the related defects and changes are observable at the end of the release. As such, for each release and file, we computed the size and the number of smells just before the first revision of a release and change metrics and defects at the last revision of the same release Figure 2. We also removed files with no churn, since they have not been touched during a release, and removed files added at the first revision of a release, since they did not have any smells before the first revision.

Table 1: Metrics, defined per file in a release, to estimate the defective files.

Metric (File C)	Description
Size	LOC
LOC_touched	Sum over revisions of LOC added+deleted+modified
NR	Number of revisions
NFix	Number of bug fixes
NAuth	Number of Authors
LOC_added	Sum over revisions of LOC added
MAX_LOC_added	Maximum over revisions of LOC added
AVG_ LOC_added	Average LOC added per revision
Churn	Sum over revisions of added – deleted LOC in C
MAX_Churn	MAX_CHURN over revisions
AVG_Churn	Average CHURN per revision
ChgSetSize	Number of files committed together with C
MAX_ChgSet	Maximum of ChgSet_Size over revisions
AVG_ChgSet	Average of ChgSet_Size over revisions
Age	Age of C in weeks
WeightedAge	Age weighted by LOC_touched
NSmells	Number of smells

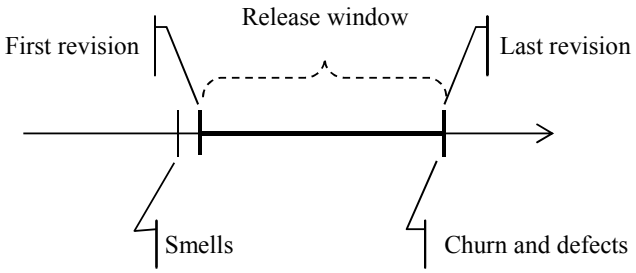


Figure 2: Data collection per release and file

We further cleaned up our database from partial data (i.e., data not containing full record of metrics). We then classified each file as smelly if it has at least one code smell and defective if it has at least one reported bug. The final dataset (A) contains 17 metrics (metrics in Table 1 plus file defectiveness) for 517 files.

Table 2 reports the proportions of files that are smelly and / or defective. Proportions can give a first insight into the relation between smelly and defective files. For example, from Table 2, practitioners can deduce that the number of defective files does not depend on being smelly as the number of defective files with or without smells is similar. Unfortunately, a simple Fisher test on the proportions reports a p-value = 0.071 (significant at 0.05) and odds ratio 1.34. Thus, as the p-value is greater than 0.05, any result drawn from Table 2 relating the number of smelly and defective files cannot be generalized.

Table 2: Proportions of files that are defective or smelly.

Smelly	Defective		Total
	No	Yes	
No	225	69	294
Yes	157	66	223
Total	382	135	517

5.4 Dataset manipulation.

Firstly, we create a dataset that has zero NSmell. To this aim, we first build a temporary dataset B+ by selecting all the smelly files in dataset A (223 files, 66 of which are defective) and then define the dataset B by setting NSmell to zero in B+. Another dataset C is created by selecting all non-smelly files from dataset A (i.e., C = A / B+). Table 3 reports the mean of the metrics computed on datasets A, B, and C. The point is now: can we perform our analysis on the synthetic dataset B as we do it on A? In other words, do datasets A and B come from the same population? Unfortunately, we do not have instruments to assert this. We can only discuss whether the two datasets are drawn from different populations.

Thus, with the Mann-Whitney test on the metrics in Table 1, we discuss the Null hypothesis that the datasets A and B come from the same population (significance threshold 0.05). The Null Hypothesis is rejected only for size (Table 3, last column). Therefore, we cannot say that the metrics computed on the two datasets come from different populations. This is the best result we could get to proceed and perform our analysis on B as we do on A.

Authorized licensed use limited to: Universita degli Studi di Roma Tor Vergata. Downloaded on April 16,2025 at 09:22:19 UTC from IEEE Xplore. Restrictions apply.

Table 3: Descriptive analysis of metrics on dataset A, B, C. *Significant at α = 0.05.

	Mean			Correlation (ρ)		MW
Variable	**A**	**B**	**C**	**NSmells**	**Defectiveness**	**A vs. B**
Size	289	271	303	0.31*	0.18*	0.027
LOC_touched	57	49	60	-0.01	0.25*	0.894
NR	9	8	10	0.02	0.38*	0.916
NFix	3	2	3	0.02	0.51*	0.515
NAuth	0.7	0.7	0.7	-0.07	0.24*	0.574
LOC_added	31	21	39	-0.04	0.18*	0.926
MAX_LOC_added	21	15	26	-0.04	0.18*	0.922
AVG_ LOC_added	12	9	14	-0.03	0.14*	0.796
Churn	16	14	17	0.01	0.12*	0.523
MAX_Churn	10	9	11	0.02	0.10*	0.476
AVG_Churn	7	7	6	0.02	0.09*	0.421
ChgSetSize	6	6	6	-0.06	0.29*	0.723
MAX_ChgSet	3	3	3	-0.05	0.19*	0.765
AVG_ChgSet	2	2	2	-0.04	0.06	0.909
Age	40	42	38	0.18*	0.02	0.084
WeightedAge	11	13	10	0.06	0.10*	0.223
NSmell	12	0	0	-	0.10*	-

All correlations between Nsmell, but Size or AGE (Table 3, fifth column), are not significant and the significant correlations are small. This is the first indication of linear independency of NSmell from the other metrics of Table 1. To further understand this point, we perform a multicollinearity test with the Variance Inflation Factor (VIF) on the regressors of a linear model that estimates defective files. VIF reports that metrics that have high multicollinearity with the other metrics (i.e., VIF > 10). These metrics are highlighted in boldface in Table 3 (first column). NSmell is not among them. As such, the information in NSmell may be considered a key variable for the estimation of the number of defective files.

As almost all the metrics in Table 1, NSmell has a significant but small correlation with file Defectiveness (Table 3, column 6). This might support the hypothesis that if any relation between a file being smelly (or having the metrics as in Table 1) and defective exists, it is rather non-linear.

5.5 Model selection.

We first selected four models widely used in defect estimation and Boolean classification [30]: Bagging, BayesNet, J48, and Logistic. We then evaluated the accuracy of each model on dataset A with the leave-one-out cross-validation approach [31]. We chose leave-one-out given that our models are trained on the entire set of data. Table 4 reports the average Precision and Recall of each model in classifying non-defective files using as input the metrics in Table 1. Table 4 shows the accuracy for the four models. Such values are in line with the results reported for the same models in literature [30]. In addition, Bagging outperforms the remaining models in both Precision and Recall, which in turn are generally better than the corresponding results for defect estimation and Boolean classification [30].

Table 4: Average accuracy of the different estimation models on dataset A.

	Bagging	**BayesNet**	**J48**	**Logistic**
Precision	0.79	0.62	0.72	0.73
Recall	0.70	0.52	0.70	0.51

5.6 Estimation results.

We applied the trained Bagging model on the dataset A, B+, B, and C. Table 4 summarizes the results. In the table, we can see the effect on the estimation of the defective files when we remove the information carried by NSmell in B (38 defective files). The drop is a substantial 42% ((66-38)/66). This means an overall reduction of 20% ((66-38)/135) in the number of defective files on the whole dataset A. This drop is not supported by any significant difference in distribution between smelly and non-smelly files (Fisher test or Mann-Whitney test in Table 2 and Table 3, resp.) or any collinearity of the other metrics with NSmell (correlation and VIF Table 3) that might have confounded the contribution of NSmell to the defectiveness estimation. Such a drop does not occur in dataset C (the subset of A in which NSmell is actually 0) either. On the one hand, this suggests that NSmell is a key estimator of file defectiveness. On the other hand, it shows practitioners a first estimation of the effect of avoiding smells still maintaining the same characteristics of the software product and development process (i.e., not changing the value of the metrics in Table 1). In our study, we do not differentiate among the types of smells that SonarQube was able to detect, nor on the types of projects we used as a testbed. Any of such differences can of course be traced in different estimation, as we briefly exemplify in Section 6. A thorough discussion of such differences is outside the scope of this paper though and will be matter of future work.

Table 5: Actual (A) and estimated (E) number of defects on the study datasets.

Dataset A		**Dataset B+**		**Dataset B**	**Dataset C**	
A	E	A	E	E	A	E
135	120	66	57	38	69	63

6 DISCUSSION

6.1 Reliability analysis

Estimations are, by nature, not perfect. Thus, our procedure entails some assumptions and the reliability of our estimation results relates to whether the assumptions hold true for the specific data analyzed. In the following, we discuss these assumptions.

Assumption 1: The estimation model is satisfactorily accurate. On whole dataset A, the Magnitude of Relative Error (MRE) is 11% ((135-120)/135), Table 5).

On dataset B+ Precision is 0.90, Recall is 0.68, whereas MRE is 14% ((66-57)/66). On dataset C, Precision is 0.95, Recall 0.98, and MRE 9% ((69-63)/69).

Overall, Precision and Recall for all datasets are in line with recent benchmarks [30] and, given that the models have been trained on a dataset of limited size from a real industrial projects, the MRE values are relatively low.

Assumption 2: The number of smells is a key estimator of file defectiveness and has little or no significant correlation with size and change metrics. Pairwise correlation analysis reports only two weak dependencies with Age and Size (Table 3) whereas multicollinearity analysis (Section 4.4) indicates NSmell is a key independent estimator.

Authorized licensed use limited to: Universita degli Studi di Roma Tor Vergata. Downloaded on April 16,2025 at 09:22:19 UTC from IEEE Xplore. Restrictions apply.

Assumption 3: Dataset B can be used for estimation. We apply our method on the dataset B as it was sampled from the same population of A. This is of course a strong assumption, that we can mitigate only in two ways: either we can pull a sample from the population of A with the characteristics of B or we are not able to reject the null hypothesis that dataset B and A come from the same population. In the latter case, we cannot exclude that the two datasets are pulled from the same population [32]. The latter is what we have tested in Section 4.

6.2 Recommendations

Our results show that avoiding smells, while keeping the same process and software characteristics, estimates a reduction in the number of defective files. However, such results must be taken with care for various reasons.

Firstly, our results are based on a What-if scenario implemented by changing specific variables' value and estimating. Although such approach is frequent in industry, it cannot be used to express any causation relation (i.e., a 20% reduction of defective files can be experienced by avoiding smells). More in general, statistical inference cannot be overestimated:

For one unfamiliar with the niceties of statistical analysis, it is difficult to view with any feeling other than awe the elaborate edifice which the authors have erected to protect their data from the cutting winds of statistical insignificance [33].

We need to consider our approach as one of the tools for decision making that must be contextualized and eventually supported by further investigation before any use. Namely, we need to be aware that

Avoiding smells does not necessarily decrease the number of defective files or may even increase it.

To illustrate what can occur by avoiding smells, we consider the case of code clones. A code clone is a portion of the code that coincides with or is very similar to another portion [34]. Clones inherently cause code bloat, thus increasing project size and maintenance costs. As such they are, in principle, smelly [1]. It is not clear though whether clones do smell in practice, in that they make the code more fault prone [35]. In our settings (i.e., avoiding clones by keeping the product and process characteristics invariant), clones and bugs are in an even more unclear relation. For example, if all clones of a buggy code M are located in the same file of M, an alternative code with no clones would not change the defectiveness of the file that includes M and avoiding code smells would not reduce the number of defective files. On the other hand, if the clones are located in different files (ripple effect [5]) and these files do not contain any other defect, avoiding code smells would reduce the number of defects assuming that the alternative code is defect free (otherwise the number can even increase).

A third reason concerns what is worth for the company. Namely,

avoiding smells is not always possible and even convenient for companies.

Clearly, there might be costs associated with an activity such as revising files to avoid code smells, setting gates to accept/reject commits that can be identified as containing smelly code. Furthermore, the correlation between smells and defects may go away without avoiding any bugs if developers are artificially induced to change their development practices. Finally, some of the smells are even unavoidable (e.g., as in the case of clones [35] [36]).

Finally, *it could be valuable to accept smells even if you know they will lead to defective files*.

In some circumstances, company managers need to trade-off between code quality and other business goals such as cost and time-to-market. Avoiding all smells might not be the primary objective of managers, but reducing the number can be a good compromise.

In conclusion, this work aims to provide actionable information to developers and managers by quantifying the consequences of accepting code with smells. The final decision of whether or not to accept code with smells depends on the specific circumstances and the type of smells under consideration.

7 Conclusions and Future Works

In this paper, we presented a method for implementing a What-if scenario analysis estimating the number of defective files in absence of smells. Our study shows that 20% of the total defective files were likely avoidable by avoiding smells. Such estimation needs to be used with the due care though as it is based on a hypothetical history (i.e., zero number of smells and same process and product change characteristics). Namely, the number of defective files can even increase for some types of smells. In addition, in some circumstances, accepting code with smells might still be a good option for a company.

We acknowledge the fact that our results can be affected by the type of product and process characteristic. Unfortunately, the choice of our metrics has been constrained by the availability of industrial data. Nevertheless, since our goal was to illustrate our method and discuss its implications for researchers and practitioners, the types of models or metrics are less relevant than the assumptions we were able to trace down for the corresponding analysis.

A what-if analysis based on scenarios simulating alternative code with no smells can eventually shed further light on our research goal. Such types of analyses are not yet commonly performed in software engineering and it would be a matter for future research. Controlled experiments can be another option (more traditional in software engineering), but their results can be limited in scope. In this case, we envision families of experiments aimed at understanding the relationships between change metrics and the existence or absence of specific smell types. This can turn out to be expensive, but it will create, in a systematic way, a body of knowledge with increasing reliability and replicability of results.

Authorized licensed use limited to: Universita degli Studi di Roma Tor Vergata. Downloaded on April 16,2025 at 09:22:19 UTC from IEEE Xplore. Restrictions apply.

ACKNOWLEDGMENT

This work has been partially supported by the GAUSS national research project (MIUR, PRIN 2015, Contract 2015KWREMX). We thank Stacy Neely for proof reading the manuscript.

REFERENCES

[1] M. Fowler, K. Beck, J. Brant, W. Opdyke, and D. Roberts, "Refactoring: Improving the Design of Existing Code," *Xtemp01*, pp. 1–337, 1999.

[2] A. Potdar and E. Shihab, "An Exploratory Study on Self-Admitted Technical Debt," in *2014 IEEE International Conference on Software Maintenance and Evolution*, 2014, pp. 91–100.

[3] G. Bavota and B. Russo, "A large-scale empirical study on self-admitted technical debt," in *Proceedings of the 13th International Workshop on Mining Software Repositories - MSR '16*, 2016, pp. 315–326.

[4] D. I. K. Sjoberg, A. Yamashita, B. C. D. Anda, A. Mockus, and T. Dyba, "Quantifying the Effect of Code Smells on Maintenance Effort," *IEEE Trans. Softw. Eng.*, vol. 39, no. 8, pp. 1144–1156, Aug. 2013.

[5] R. Fanta and V. Rajlich, "Removing clones from the code," *J. Softw. Maint. Res. Pract.*, vol. 11, no. 4, pp. 223–243, 1999.

[6] E. Murphy-Hill, C. Parnin, and A. P. Black, "How We Refactor, and How We Know It," *IEEE Trans. Softw. Eng.*, vol. 38, no. 1, pp. 5–18, Jan. 2012.

[7] W. Li and R. Shatnawi, "An empirical study of the bad smells and class error probability in the post-release object-oriented system evolution," *J. Syst. Softw.*, vol. 80, pp. 1120–1128, 2007.

[8] A. Yamashita, "Assessing the capability of code smells to explain maintenance problems: An empirical study combining quantitative and qualitative data," *Empir. Softw. Eng.*, vol. 19, no. 4, pp. 1111–1143, 2014.

[9] M. Abbes, F. Khomh, Y. G. Guéhéneuc, and G. Antoniol, "An empirical study of the impact of two antipatterns, Blob and Spaghetti Code, on program comprehension," in *Proceedings of the European Conference on Software Maintenance and Reengineering, CSMR*, 2011, pp. 181–190.

[10] I. Deligiannis, I. Stamelos, L. Angelis, M. Roumeliotis, and M. Shepperd, "A controlled experiment investigation of an object-oriented design heuristic for maintainability," *J. Syst. Softw.*, vol. 72, no. 2, pp. 129–143, 2004.

[11] A. Yamashita and L. Moonen, "To what extent can maintenance problems be predicted by code smell detection? -An empirical study," *Inf. Softw. Technol.*, vol. 55, pp. 2223–2242, 2013.

[12] P. Tomas, M. J. Escalona, and M. Mejias, "Open source tools for measuring the Internal Quality of Java software products. A survey," *Comput. Stand. Interfaces*, vol. 36, no. 1, pp. 244–255, Nov. 2013.

[13] F. D. Davis, J. E. Kottemann, and W. E. Remus, "What-if analysis and the illusion of control," in *Twenty-Fourth Annual Hawaii International Conference on System Sciences*, 1991.

[14] G. Bavota, R. Oliveto, M. Gethers, D. Poshyvanyk, and A. De Lucia, "Methodbook: Recommending move method refactorings via relational topic models," *IEEE Trans. Softw. Eng.*, vol. 40, no. 7, pp. 671–694, 2014.

[15] D. Falessi and A. Voegele, "Validating and prioritizing quality rules for managing technical debt: An industrial case study," in *2015 IEEE 7th International Workshop on Managing Technical Debt, MTD 2015 - Proceedings*, 2015, pp. 41–48.

[16] I. Griffith and C. Izurieta, "Design Pattern Decay: The Case for Class Grime," in *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2014, p. 39:1--39:4.

[17] T. Mens and T. Tourwe, "A survey of software refactoring," *IEEE Trans. Softw. Eng.*, vol. 30, 2004.

[18] M. Kim, T. Zimmermann, and N. Nagappan, "An Empirical Study of Refactoring Challenges and Benefits at Microsoft," *IEEE Trans. Softw. Eng.*, pp. 1–1, Apr. 2014.

[19] J. Ratzinger, T. Sigmund, and H. C. Gall, "On the relation of refactorings and software defect prediction," in *Proceedings of the 2008 international workshop on Mining software repositories - MSR '08*, 2008, p. 35.

[20] D. Steidl and S. Eder, "Prioritizing maintainability defects based on refactoring recommendations," in *Proceedings of the 22nd International Conference on Program Comprehension - ICPC 2014*, 2014, pp.168–76.

[21] F. A. Fontana, M. Zanoni, A. Marino, and M. V. M??ntyl??, "Code smell detection: Towards a machine learning-based approach," in *IEEE International Conference on Software Maintenance, ICSM*, 2013, pp. 396–399.

[22] F. A. Fontana, P. Braione, and M. Zanoni, "Automatic detection of bad smells in code: An experimental assessment," *J. Object Technol.*, vol. 11, 2012.

[23] N. Moha and Y. Guéhéneuc, "DECOR: A method for the specification and detection of code and design smells," *... , IEEE Trans.*, vol. 36, pp. 20–36, 2010.

[24] N. Zazworka, R. O. Spínola, A. Vetro', F. Shull, and C. Seaman, "A case study on effectively identifying technical debt," in *Proceedings of the 17th International Conference on Evaluation and Assessment in Software Engineering - EASE '13*, 2013, p. 42.

[25] F. Palomba, M. D. P. Gabriele Bavota, R. Oliveto, and A. De Lucia, "Do they Really Smell Bad? A Study on Developers' Perception of Code Bad Smells," in *30th International Conference on Software Maintenance and Evolution (ICSM 2014)*, 2014.

[26] R. Kazman, Y. Cai, R. Mo, Q. Feng, L. Xiao, S. Haziyev, V. Fedak, and A. Shapochka, "A Case Study in Locating the Architectural Roots of Technical Debt," in *IEEE/ACM 37th IEEE International Conference on Software Engineering*, 2015, pp. 179–188.

[27] H. Liu, Z. Ma, W. Shao, and Z. Niu, "Schedule of bad smell detection and resolution: A new way to save effort," *IEEE Trans. Softw. Eng.*, vol. 38, no. 1, pp. 220–235, 2012.

[28] C. Bird and T. Zimmermann, "Assessing the Value of Branches with What-if Analysis," *Proc. ACM SIGSOFT 20th Int. Symp. Found. Softw. Eng.*, p. 45:1-45:11, 2012.

[29] M. D'Ambros, M. Lanza, and R. Robbes, "Evaluating defect prediction approaches: a benchmark and an extensive comparison," *Empirical Software Engineering*, vol. 17. pp. 531–577, 2012.

[30] S. Kim, H. Zhang, R. Wu, and L. Gong, "Dealing with noise in defect prediction," in *Proceeding of the 33rd international conference on Software engineering - ICSE '11*, 2011, p. 481.

[31] I. H. Witten and E. Frank, *Data Mining: Practical Machine Learning Tools and Techniques, Second Edition*. Morgan Kaufmann Series in Data Management Systems, 2005.

[32] M. P. Fay and M. A. Proschan, "Wilcoxon-Mann-Whitney or t-test? On assumptions for hypothesis tests and multiple interpretations of decision rules," *Stat. Surv.*, vol. 4, no. 0, pp. 1–39, 2010.

[33] E. E. Grant and H. Sackman, "An exploratory investigation of programmer performance under on-line and off-line conditions," *IEEE Trans. Hum. factors Electron.*, vol. 1, 1967.

[34] T. Kamiya, S. Kusumoto, and K. Inoue, "CCFinder: A multilinguistic token-based code clone detection system for large scale source code," *IEEE Trans. Softw. Eng.*, vol. 28, no. 7, pp. 654–670, 2002.

[35] F. Rahman, C. Bird, and P. Devanbu, "Clones: what is that smell?," *Empir. Softw. Eng.*, vol. 17, no. 4–5, pp. 503–530, Dec. 2011.

[36] C. J. Kapser and M. W. Godfrey, "'cloning considered harmful' considered harmful: Patterns of cloning in software," *Empir. Softw. Eng.*, vol. 13, no. 6, pp. 645–692, 2008.

Authorized licensed use limited to: Universita degli Studi di Roma Tor Vergata. Downloaded on April 16,2025 at 09:22:19 UTC from IEEE Xplore. Restrictions apply.